The Architecture of drnealaggarwal.info

A visual teardown of a pure Python, flat-file web application.

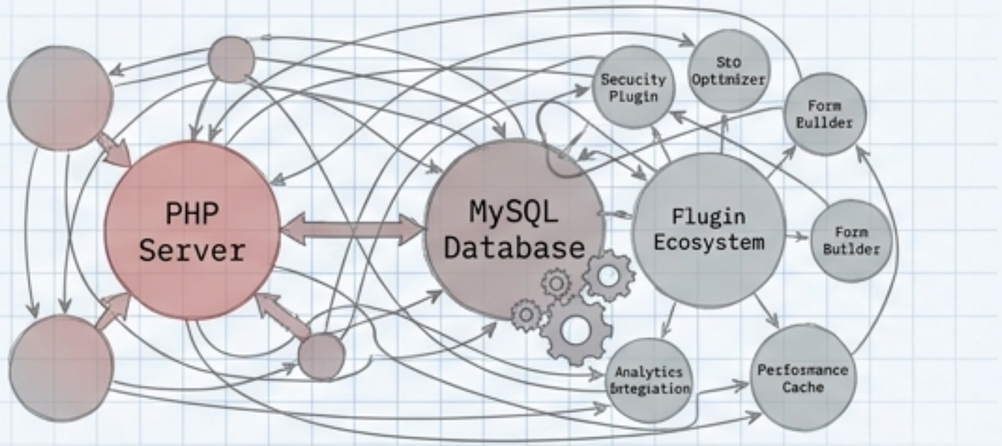
Rendered in 13 structural schematics.

app.py

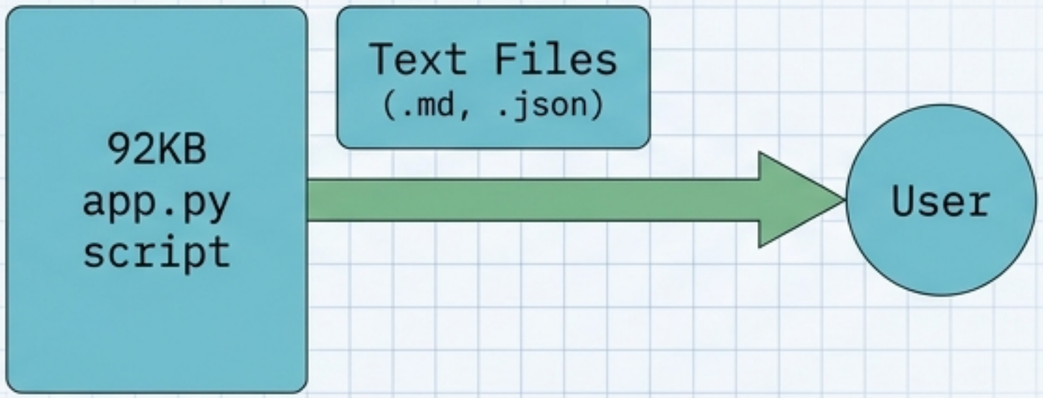
NotebookLM

The No-Database Manifesto

The Standard Stack (e.g., WordPress)



Dr. Neal's Stack

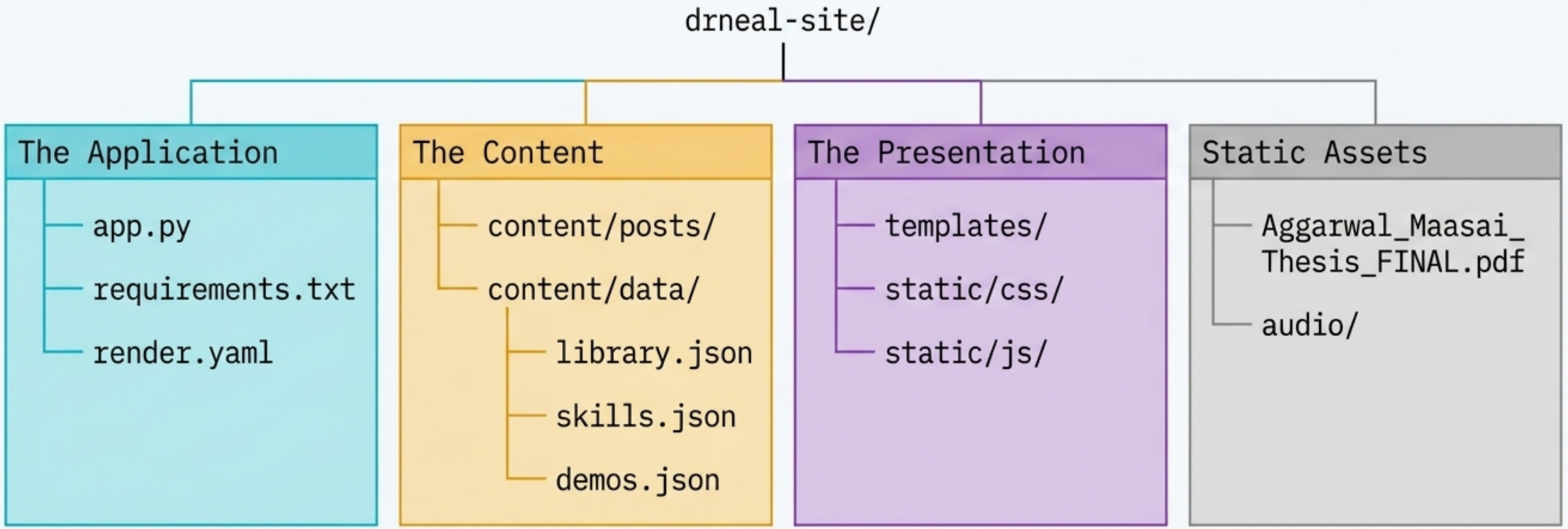


Data Storage	Relational Database (MySQL)	Data DB Storage	Plain Text Files (.md, .json)
Rendering Speed	Blocking DB Queries	Rendering Dictionant	In-memory Dictionary Lookups
Maintenance	Constant security/plugin updates	Zero-maintenance proceses	Zero-maintenance pure Python
Version Control	Messy database dumps	Version Control	100% Git versioned history

Zero databases, **zero** CMS, **no bloat**.
Just a **92KB** Python file, Markdown, JSON, and Flask.

NotebookLM

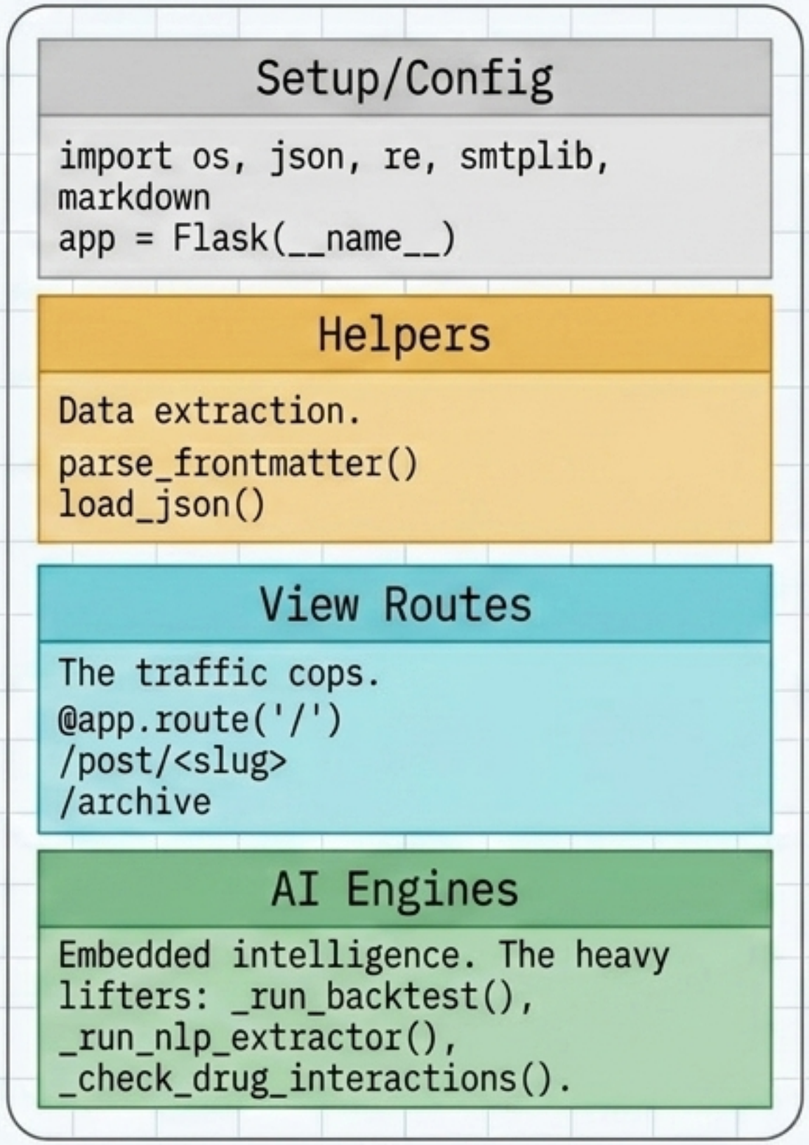
System Architecture: The Repository Map



A strict separation of concerns: logic lives in the root, prose lives in Markdown, structured lists live in JSON, and presentation logic is isolated in HTML templates.

NotebookLM

Anatomy of a 92KB Monolith: inside app.py

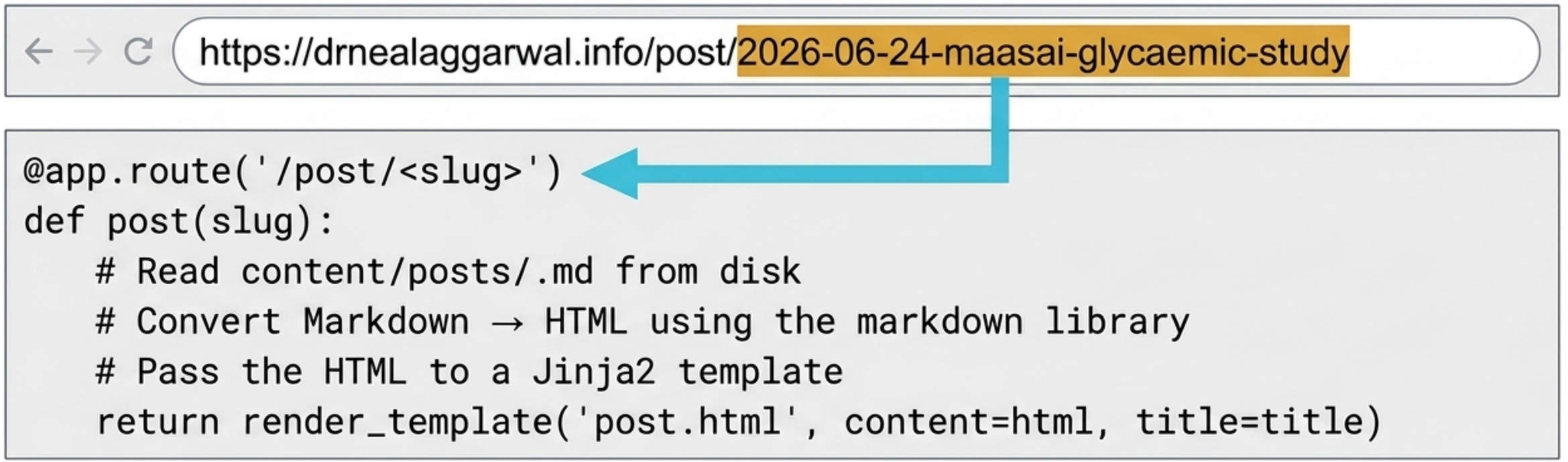


Secret key generation via environment variables.

Treating app.py not as a simple script, but as a heavily compartmentalized engine where routing, file parsing, and machine learning exist in a single namespace.

NotebookLM

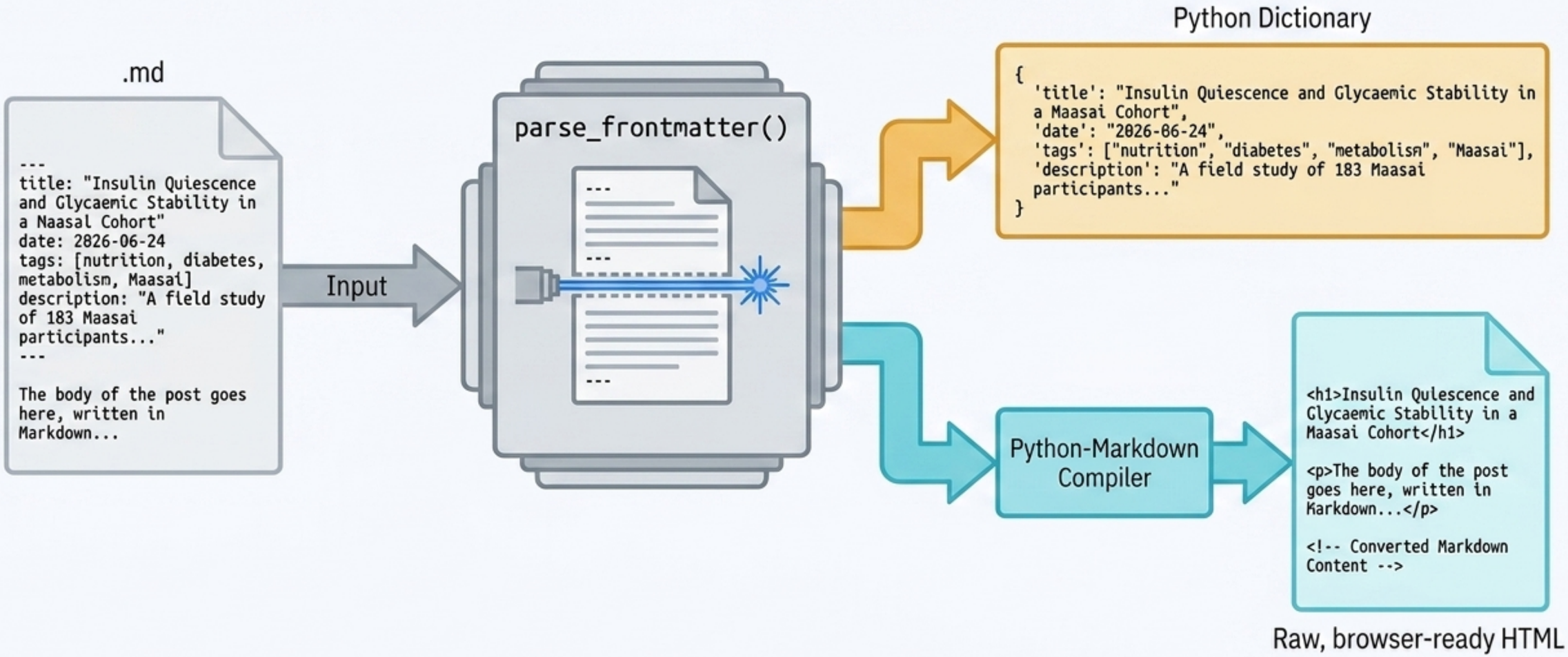
The Routing Engine: Mapping the Web to Python



Flask's routing engine uses variable capture (<slug>). It snatches the exact post name from the URL and passes it directly into the Python function as an argument, triggering the file-read process.

NotebookLM

The Content Pipeline: Parsing Markdown



NotebookLM

The Data Pipeline: Structured Context via JSON

By utilizing JSON for lists and collections, the site achieves the structured querying of a database without the latency of SQL connections.

NotebookLM

The Three Data Substrates

Prose (.md)

- **Usage:** Dynamic, long-form content (Blog Posts).
- **Processing:** Split into metadata (YAML) and compiled to HTML via the Markdown library.
- **Location:** `content/posts/`

Structured Lists (.json)

- **Usage:** Repeating collections (Library, Skills, Demos).
- **Processing:** Parsed into Python dictionaries for programmatic filtering and looping.
- **Location:** `content/data/`

Static Assets (.pdf, .css, .m4a)

- **Usage:** Unprocessed pass-throughs.
- **Processing:** Zero processing. Served directly to the browser via the `/static/` prefix.
- **Location:** `static/`

NotebookLM

The Template Layer: Jinja2 Injection

templates/post.html

```
<!-- templates/post.html -->
<h1>{{ title }}</h1>
<p class="date">{{ date }}</p>

<div class="content">
    {{ content | safe }}
</div>
```

app.py (Python Variables)

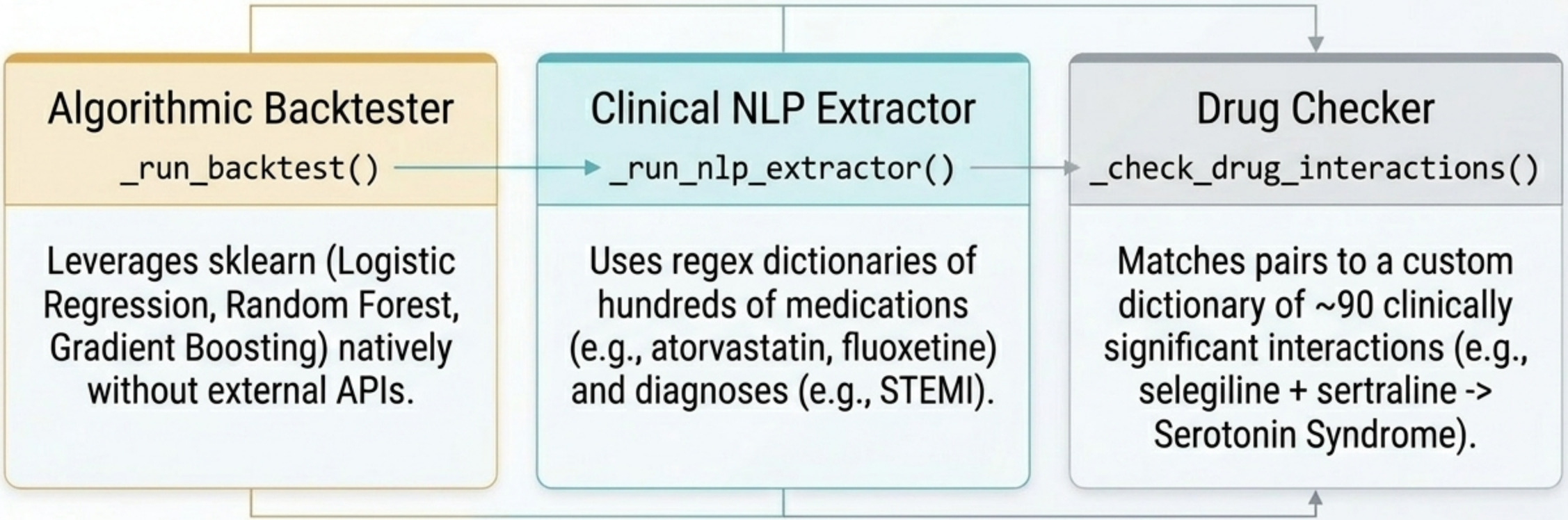
```
title = "Insulin Quiescence…"
date = "2026-06-24"
content = "<p>A field study…</p>"
```

`{{ content | safe }}`

The | safe filter is critical. It commands Jinja2 not to escape the characters, allowing the pre-compiled Markdown HTML to render correctly.

NotebookLM

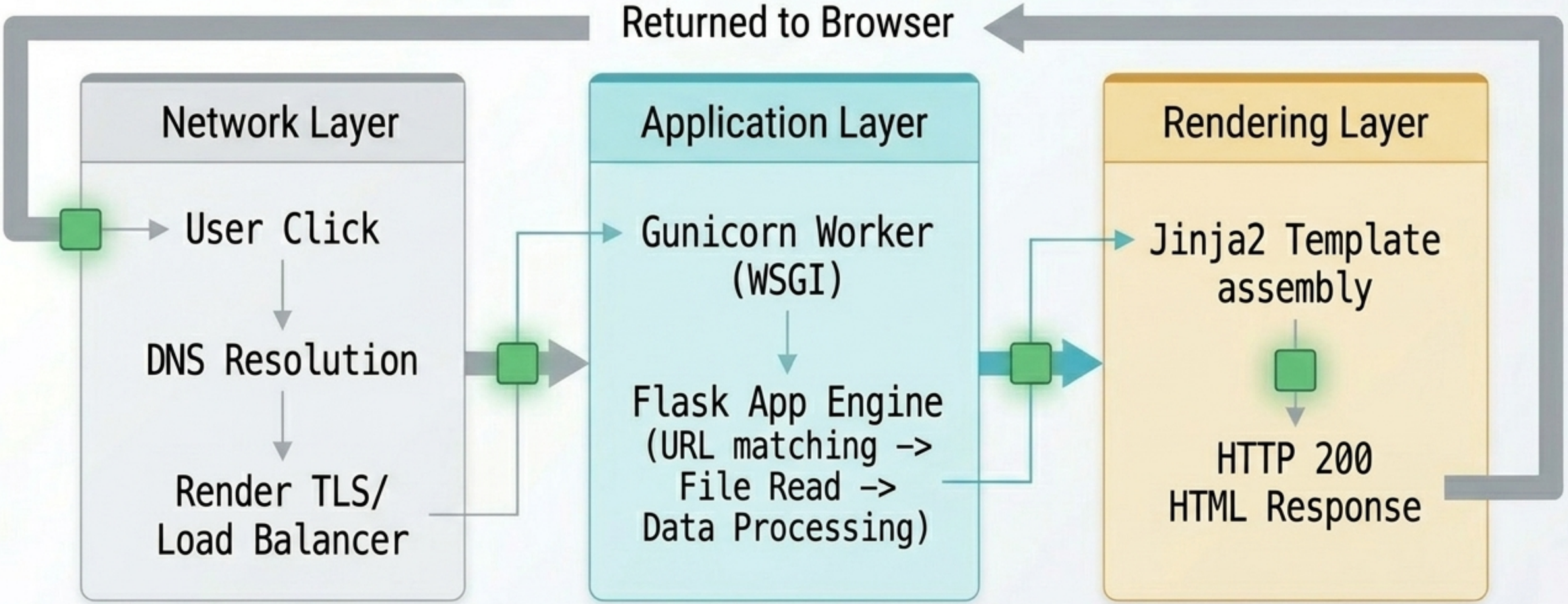
Embedded Native Microservices



By embedding these engines directly as pure-Python functions inside `app.py`, the site bypasses external API latency entirely, executing complex logic natively.

NotebookLM

The Request Lifecycle



NotebookLM

Production Deployment Pipeline

The Push

`Git push` to `master`.

The Webhook

Render receives the trigger instantly.

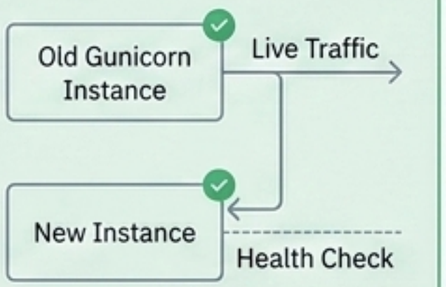
The Build

Executes `pip install -r requirements.txt` (installing Flask 3.0+, Gunicorn, Markdown).

The Spin-Up

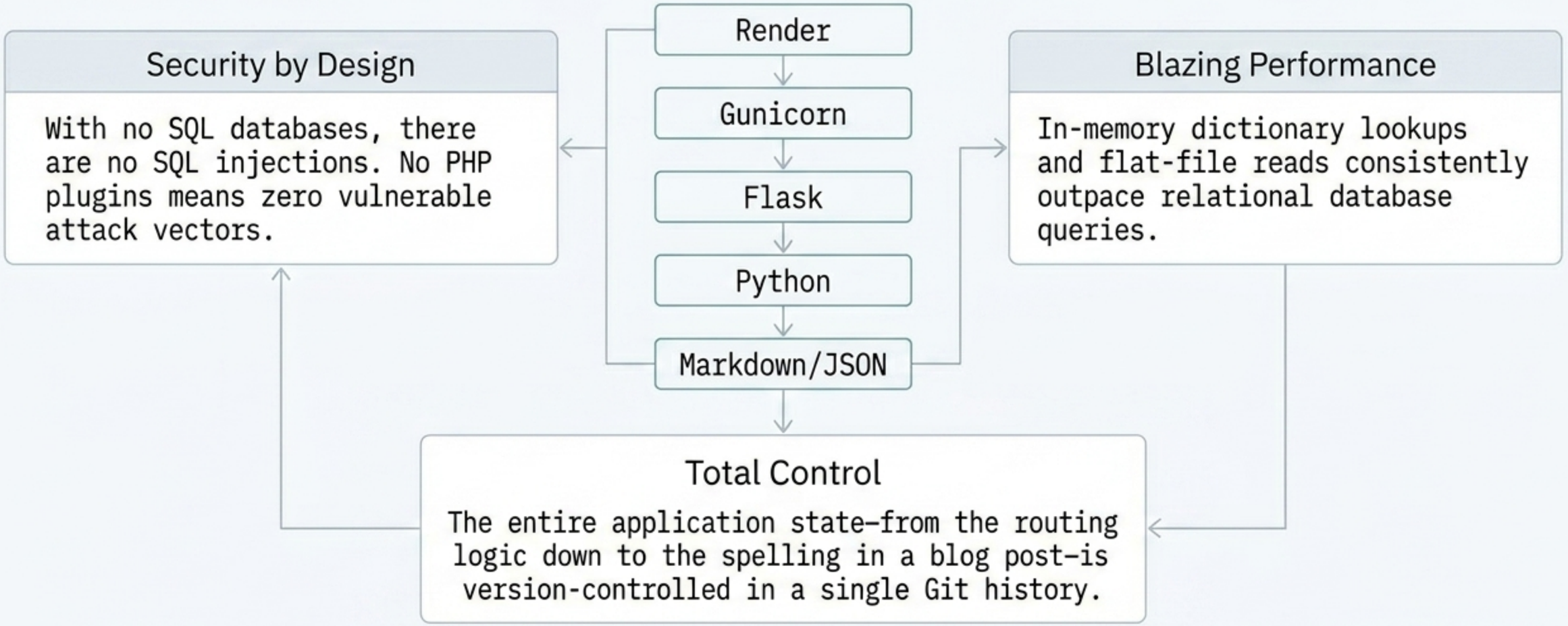
```
services:
  - type: web
    name: drneal-site
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app
    envVars:
      - key: SECRET_KEY
        generateValue: true
```

Zero-Downtime Swap



NotebookLM

The Elegance of Simplicity



A technical artifact that serves as both a portfolio and a testament to principled engineering.

NotebookLM